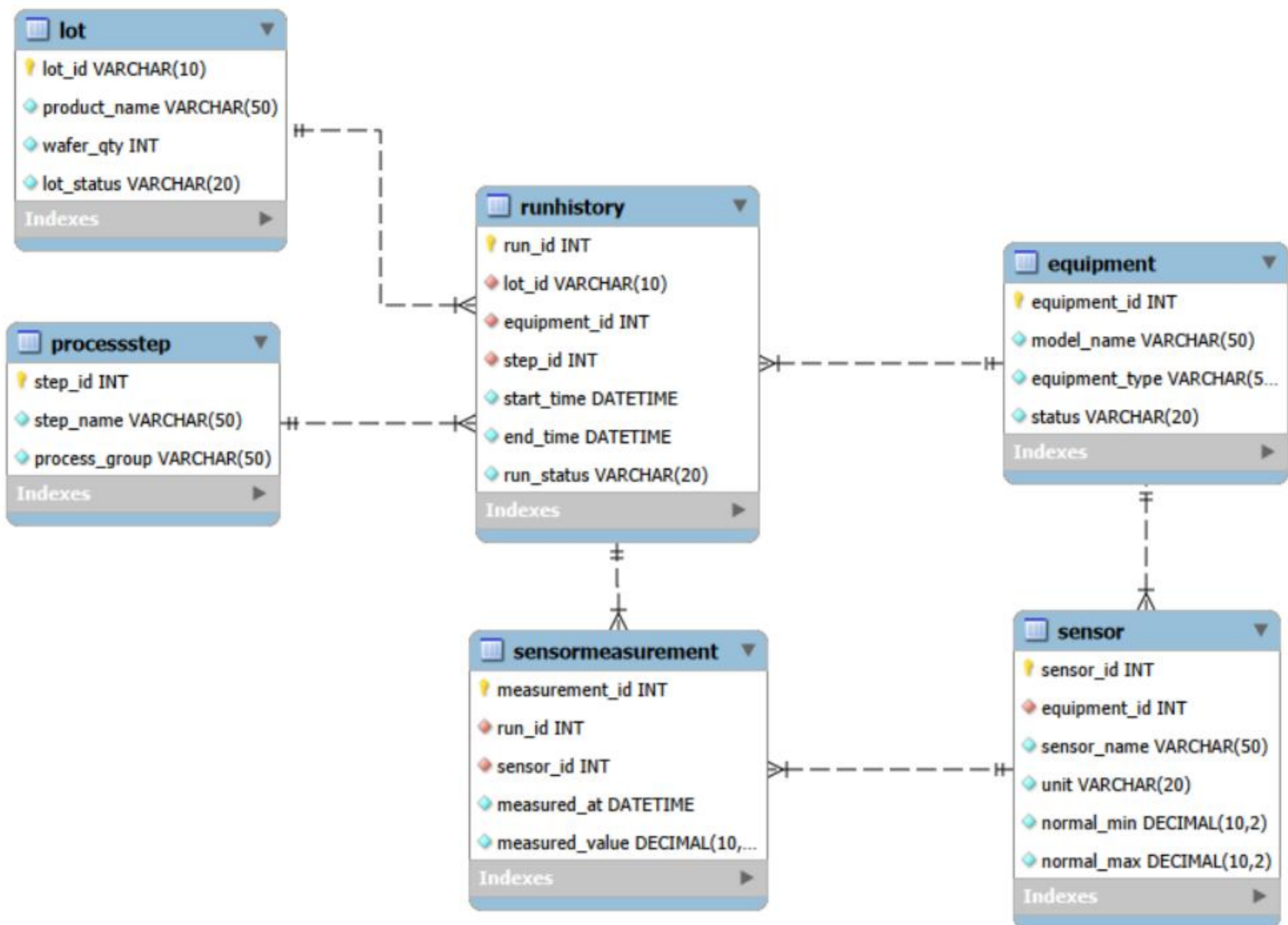


반도체장비데이터관리 11

Python 연계 데이터 처리



The slide features several decorative squares: a small light gray square at the top left, a medium light gray square at the top right, a medium light gray square on the middle left, and a small dark blue square on the middle right.

1) Python과 DB 연결

실습 환경 설정하기

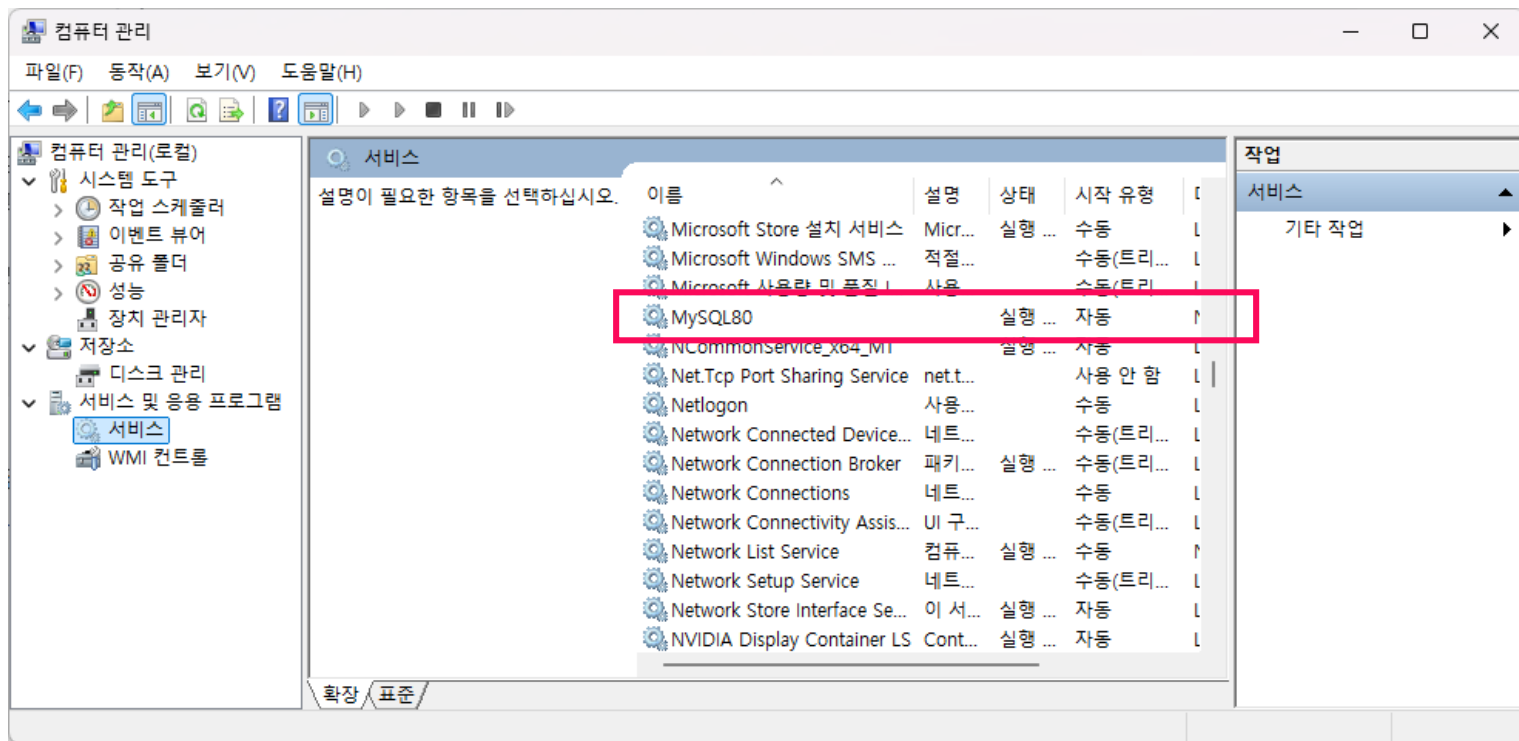
- 실습에서 사용할 프로그램

프로그램	용도
MySQL Server	데이터베이스 실행
MySQL Workbench	DB 확인 및 SQL 실행
Python	코드 실행
VS Code	Python 개발 환경
Python 확장	VS Code에서 Python 실행 지원

실습 환경 설정하기

1단계 SQL 서버가 작동 중인지 확인하기

- MySQL Workbench에서 간단한 쿼리를 실행하여 확인
- 또는 [컴퓨터 관리 > 서비스 및 응용 프로그램 > 서비스] 에서 MySQL이 실행 중인지 확인



실습 환경 설정하기

2단계 Python 설치

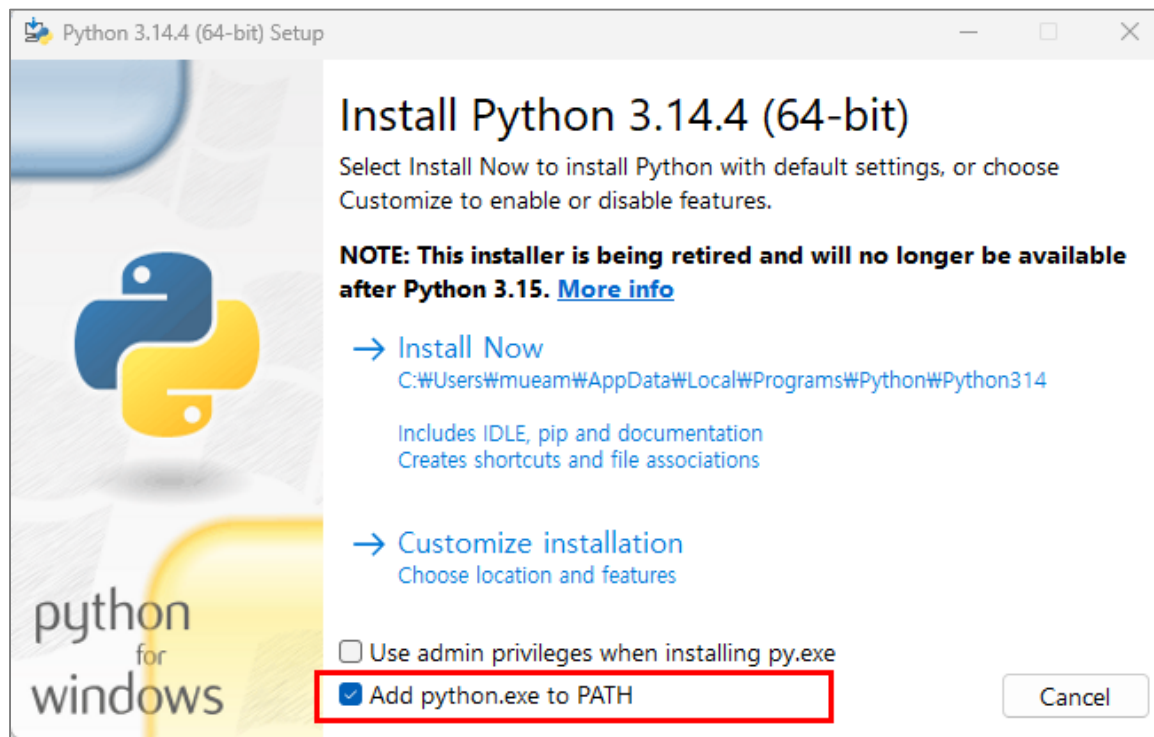
- Python가 설치되어 있는지 확인하기

- 명령 프롬프트에 다음을 입력 →

```
C:\Users\mueam>python --version  
Python 3.14.4
```

- 설치되어 있지 않다면 설치

- 이때 반드시 다음 항목에 체크 →



실습 환경 설정하기

3단계 VS Code 설치 및 설정

- VS Code 설치 후 Python 확장 설치
- 본인의 코드를 저장할 폴더를 하나 생성한 후, 이를 프로젝트 폴더로 설정
- Terminal에서 다음을 입력하여 필요한 라이브러리 설치

→ `PS D:\semicon_project> pip install pymysql pandas matplotlib`

- 설치하는 라이브러리 : pymysql, pandas, matplotlib
- pymysql은 파이썬에서 MySQL을 쓸 수 있게 해주는 라이브러리

실습 환경 설정하기

4단계 DB 연결 테스트 코드 작성

- 프로젝트 폴더 내에 test.py 파일 생성
- 다음 예제 코드 작성하기 →
user='root',
password='1234',
→ 이 부분은 본인 계정의 ID/PW로 입력해야 함!

```
import pymysql
import pandas as pd

# 1. DB 연결
conn = pymysql.connect(
    host='localhost',
    user='root',
    password='1234',
    database='ProcessSensorDB',
    charset='utf8mb4'
)

# 2. SQL 쿼리 작성
query = """
SELECT *
FROM SensorMeasurement;
"""

# 3. SQL 실행 결과를 DataFrame으로 가져오기
df = pd.read_sql(query, conn)

# 4. 결과 확인
print(df.head())

# 5. DB 연결 종료
conn.close()
```


실습 환경 설정하기

5단계 작성한 코드 실행하여 결과 확인

- 실행방법 1: VS Code 오른쪽 상단에서 다음 버튼을 클릭 →



- 실행방법 2: 터미널에서 다음을 입력

```
python db_test.py
```

또는

```
py db_test.py
```

- 다음과 같은 실행 결과가 출력되면 SQL 서버와의 연결이 성공한 것

	measurement_id	run_id	sensor_id	measured_at	measured_value
0	1	1	1	2024-03-15 09:05:00	398.00
1	2	1	1	2024-03-15 09:15:00	402.00
2	3	1	1	2024-03-15 09:25:00	405.00
3	4	1	2	2024-03-15 09:05:00	1.95
4	5	1	2	2024-03-15 09:15:00	2.00

자주 발생하는 오류

1. `ModuleNotFoundError: No module named 'pymysql'`

라이브러리가 설치되지 않은 상태입니다.

2. `Access denied for user 'root'`

MySQL 비밀번호가 틀렸을 가능성이 큼니다.

3. `Unknown database 'ProcessSensorDB'`

DB 이름이 다르거나 생성되지 않은 상태입니다.

DB 연결하는 가장 기본적인 코드

0. 필요한 라이브러리 불러오기

```
import pymysql
import pandas as pd
```

- pymysql : Python에서 MySQL 데이터베이스에 접속하기 위한 라이브러리
- pandas : SQL 실행 결과를 표 형태의 데이터인 DataFrame으로 다루기 위해 사용

DB 연결하는 가장 기본적인 코드

1. DB 연결하기

- `pymysql.connect()` : Python에서 MySQL 데이터베이스에 접속하는 함수

1. DB 연결

```
conn = pymysql.connect(  
    host='localhost',  
    user='root',  
    password='1234',  
    database='ProcessSensorDB',  
    charset='utf8mb4'  
)
```

항목	의미
host	DB가 실행 중인 위치
user	DB 사용자 계정
password	DB 비밀번호
database	사용할 데이터베이스 이름
charset	한글 데이터 처리를 위한 문자 인코딩

DB 연결하는 가장 기본적인 코드

2. SQL 쿼리 작성하기

- 실행할 SQL 문을 문자열로 작성함
- Python에서는 SQL 문을 직접 실행하는 것이 아니라, 먼저 문자열 형태로 저장한 뒤 DB에 전달함

```
# 2. SQL 쿼리 작성
query = """
SELECT *
FROM SensorMeasurement;
"""
```

DB 연결하는 가장 기본적인 코드

3. SQL 실행 결과를 DataFrame으로 가져오기

- `pd.read_sql()` : SQL 실행 결과를 pandas DataFrame으로 가져오는 함수
 - `conn` : DB 연결 정보
 - `query` : SQL 쿼리를 나타내는 문자열

```
# 3. SQL 실행 결과를 DataFrame으로 가져오기  
df = pd.read_sql(query, conn)
```

DB 연결하는 가장 기본적인 코드

4. 결과 확인하기

- `df.head()` : DataFrame의 앞부분 5개 행을 확인하는 함수

```
# 4. 결과 확인  
print(df.head())
```

5. 데이터베이스 연결 해제하기

```
# 5. DB 연결 종료  
conn.close()
```

파이썬에서 쿼리하기 연습 문제

1. ProcessStep 테이블의 모든 데이터를 조회하시오.
2. Equipment 테이블에서 다음 컬럼만 조회하시오.
 - model_name, status
3. 현재 상태(status)가 'active'인 장비만 조회하시오.

파이썬에서 쿼리하기 연습 문제

4. run_id가 1인 측정 데이터만 조회하시오.
5. RunHistory 테이블의 데이터를 start_time 기준으로 오래된 순서대로 정렬하여 조회하시오.
6. ProcessStep 테이블에서 process_group이 'ETCH'인 공정 단계만 조회하시오.

파이썬에서 쿼리하기 연습 문제

7. RunHistory와 Equipment 테이블을 연결하여 공정 실행 이력과 장비 정보를 함께 조회하시오.
8. 각 장비가 몇 번의 공정을 수행했는지 조회하시오.
9. 장비별로 정상 범위를 벗어난 측정값이 몇 번 발생했는지 조회하시오.

The background features several decorative squares: a small grey square at the top left, a larger grey square at the top right, a medium grey square on the left side, and a small dark blue square on the right side.

2) 사용자 입력값을 SQL에 넣기

왜 필요한가?

- 지금까지는 SQL 조건을 코드 안에 직접 썼다.
 - 매번 run_id 값을 바꾸려면 코드를 직접 수정해야 함

```
query = """
SELECT *
FROM SensorMeasurement
WHERE run_id = 1;
"""
```

→ 사용자에게 값을 입력받아 SQL 조건에 넣을 수 있다.

위험한 방식: 파이썬 문법의 format 사용

```
run_id = input("조회할 run_id를 입력하세요: ")

query = f"""
SELECT *
FROM SensorMeasurement
WHERE run_id = {run_id};
"""
```

- 이렇게 쓰면 위험한 이유
 - 사용자가 입력한 값이 SQL 문장에 그대로 들어가기 때문
 - 잘못된 입력이나 악의적인 입력이 들어갈 수 있음

안전한 방식: 파라미터 바인딩(%s) 사용

```
run_id = input("조회할 run_id를 입력하세요: ")

query = """
SELECT *
FROM SensorMeasurement
WHERE run_id = %s;
"""

df = pd.read_sql(query, conn, params=(run_id,))
```

- %s: 나중에 실제 값이 들어갈 자리
- SQL 문 안에 값을 붙이지 않고 params를 통해 전달

- 파라미터 바인딩을 사용하면,
 - 외부 입력을 SQL 문과 분리하여 안전하게 처리
 - 동일한 쿼리의 재사용성을 높여 서버 부하를 줄임

파라미터 바인딩

- 개념

- 외부(클라이언트, API 요청 등)에서 전달된 데이터를
- 함수 매개변수나 SQL 쿼리의 특정 변수 위치에 동적으로 연결 및 매핑하는 기술

- 구조

```
query = """
SELECT *
FROM 테이블명
WHERE 컬럼명 = %s;
"""

df = pd.read_sql(query, conn, params=(값,))
```

- 주의할 점: params의 값이 하나일 때도 쉼표가 필요하다.

파라미터 바인딩 예제

문제: 사용자로부터 run_id를 입력 받아 해당 Run의 센서 측정 데이터를 조회하는 프로그램을 작성하시오.

```
run_id = input("조회할 run_id를 입력하세요: ")

query = """
SELECT *
FROM SensorMeasurement
WHERE run_id = %s;
"""

df = pd.read_sql(query, conn, params=(run_id,))
print(df)
```

- 사용자가 입력한 run_id 값이 %s 자리에 들어가서 SQL이 실행됨
- 즉, 코드를 고치지 않고 입력값만 바꾸어 원하는 Run 데이터를 조회할 수 있음

파라미터 바인딩 예제

문제: 사용자로부터 run_id를 입력 받아 해당 Run의 센서 측정 데이터를 조회하는 프로그램을 작성하시오.

```
run_id = input("조회할 run_id를 입력하세요: ")

query = """
SELECT *
FROM SensorMeasurement
WHERE run_id = %s;
"""

df = pd.read_sql(query, conn, params=(run_id,))
print(df)
```

- 사용자가 입력한 run_id 값이 %s 자리에 들어가서 SQL이 실행됨
- 즉, 코드를 고치지 않고 입력값만 바꾸어 원하는 Run 데이터를 조회할 수 있음

파라미터 바인딩 연습 문제

연습 문제 1. 사용자로부터 조회할 sensor_name을 입력 받아
해당 센서의 측정 데이터를 조회하는 프로그램을 작성하시오.

조회할 항목:

- measurement_id, run_id, sensor_name, measured_value, unit

조건:

- SensorMeasurement와 Sensor 테이블을 JOIN할 것
- run_id 사용자 입력값은 %s를 사용하여 전달할 것

파라미터 바인딩 연습 문제

연습 문제 2. 사용자로부터 equipment_id를 입력 받아,
해당 장비에서 수행된 공정 실행 이력을 조회하는 프로그램을 작성하시오.

조회할 항목:

- run_id, equipment_id, model_name, step_name,
- start_time, end_time, run_status

조건:

- RunHistory, Equipment, ProcessStep 테이블을 JOIN할 것
- 사용자 입력값은 %s를 사용하여 SQL 조건에 전달할 것
- 실행 시작 시간 기준으로 오름차순 정렬할 것

파라미터 바인딩 연습 문제

연습 문제 3. 사용자로부터 lot_id를 입력 받아,
해당 Lot이 어떤 공정 단계를 거쳤는지 조회하는 프로그램을 작성하시오.

조회할 항목:

- lot_id, product_name, run_id, step_name, process_group,
- equipment_id, start_time, run_status

조건:

- Lot, RunHistory, ProcessStep, Equipment 테이블을 JOIN할 것
- 사용자 입력값은 %s를 사용하여 SQL 조건에 전달할 것
- 실행 시작 시간 기준으로 오름차순 정렬할 것



3) 원시 데이터의 이해

원시 데이터의 특징

- ① 하나의 Run 정보가 여러 줄로 나뉘어 저장됨
→ 즉, 하나의 공정 실행 정보가 여러 행에 분산됨
- ② 센서 종류와 측정값이 세로 형태(Long Format)로 저장됨
→ 센서 종류가 컬럼이 아니라 데이터 값으로 저장됨

measurement_id	run_id	sensor_id	measured_at	measured_value
1	1	1	2024-03-15 09:05:00	398.00
2	1	1	2024-03-15 09:15:00	402.00
3	1	1	2024-03-15 09:25:00	405.00
4	1	2	2024-03-15 09:05:00	1.95
5	1	2	2024-03-15 09:15:00	2.00
6	1	2	2024-03-15 09:25:00	2.05
7	2	3	2024-03-15 10:05:00	500.00
8	2	3	2024-03-15 10:15:00	508.00

원시 데이터의 특징

- ③ 다양한 센서 데이터가 하나의 테이블에 함께 저장됨
 - Temperature, Pressure, GasFlow 등이 함께 저장됨
- ④ 저장 중심 구조로 설계되어 있음
 - 데이터 기록과 관리에는 유리하지만 분석과 비교에는 불편할 수 있음

measurement_id	run_id	sensor_id	measured_at	measured_value
1	1	1	2024-03-15 09:05:00	398.00
2	1	1	2024-03-15 09:15:00	402.00
3	1	1	2024-03-15 09:25:00	405.00
4	1	2	2024-03-15 09:05:00	1.95
5	1	2	2024-03-15 09:15:00	2.00
6	1	2	2024-03-15 09:25:00	2.05
7	2	3	2024-03-15 10:05:00	500.00
8	2	3	2024-03-15 10:15:00	508.00

원시 데이터의 특징으로 인해 발생하는 어려움

- ① Run별 상태를 한눈에 보기 어렵다
 - 하나의 Run 정보가 여러 줄에 나뉘어 있음
- ② 센서값 비교가 불편하다
 - Temperature와 Pressure가 같은 행에 없음
- ③ 특정 센서만 분석하려면 별도로 분리해야 한다
 - 원하는 센서 데이터를 먼저 필터링해야 함
- ④ 그래프 및 통계 분석에 바로 사용하기 어렵다
 - 분석 전에 데이터 구조를 정리해야 함

원시 데이터의 특징으로 인해 발생하는 어려움

⑤ 분석용 표 형태와 구조가 다르다

[원시 데이터]

run_id	sensor_name	measured_value
1	Temperature	85
1	Pressure	3.2
1	GasFlow	120
2	Temperature	88
2	Pressure	3.5
2	GasFlow	118

- Long Format

- : 데이터를 세로로 길게 쌓은 형태
- : 저장과 기록에 유리

[분석용 데이터]

run_id	Temperature	Pressure	GasFlow
1	85	3.2	120
2	88	3.5	118

- Wide Format

- : 하나의 관측 대상(행)에 대한 여러 변수(열)를 가로로 길게 나열하는 방식
- : 비교, 통계, 시각화에 유리

**데이터를 분석하기 전에
필요한 형태로 정리하는 과정이 필요하다.**

The slide features several decorative squares: a small grey square at the top left, a medium grey square at the bottom left, a medium grey square at the top right, and a small dark blue square at the bottom right.

4) 데이터 정제하기

데이터 정제 Overview

- 실습 목표
 - Python과 Pandas를 이용하여 DB의 원시 센서 데이터를 분석 가능한 형태로 변환한다.
- 주요 흐름
 - ① 원시 데이터 조회
 - ② 필요한 데이터 선택
 - ③ 조건에 맞게 정리
 - ④ 데이터 묶기(groupby)
 - ⑤ 형태 변환(pivot)
 - ⑥ 분석용 데이터셋 생성

① 원시 데이터 조회

- Sensor와 SensorMeasurement를 JOIN하여 센서 측정 데이터를 조회한다.

```
query = """
SELECT
    *
FROM SensorMeasurement sm
JOIN Sensor s
ON sm.sensor_id = s.sensor_id;
"""

df = pd.read_sql(query, conn)

print(df.head())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 36 entries, 0 to 35
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   measurement_id        36 non-null    int64
1   run_id                 36 non-null    int64
2   sensor_id              36 non-null    int64
3   measured_at            36 non-null    datetime64[us]
4   measured_value         36 non-null    float64
5   sensor_id              36 non-null    int64
6   equipment_id           36 non-null    int64
7   sensor_name            36 non-null    str
8   unit                   36 non-null    str
9   normal_min             36 non-null    float64
10  normal_max             36 non-null    float64
dtypes: datetime64[us](1), float64(3), int64(5), str(2)
memory usage: 3.2 KB
```

② 필요한 데이터 선택

- 분석에 필요한 데이터만 선택한다.

```
df = df[['run_id', 'sensor_name', 'measured_value']]  
  
print(df.head())
```

	run_id	sensor_name	measured_value
0	1	chamber_temp	398.0
1	1	chamber_temp	402.0
2	1	chamber_temp	405.0
3	4	chamber_temp	399.0
4	4	chamber_temp	401.0

③ 조건 필터링

- 특정 조건에 해당하는 데이터만 선택한다.

- 예시 1. 특정 센서만 추출

```
temp_df = df[df['sensor_name'] == 'chamber_temp']  
print(temp_df)
```

	run_id	sensor_name	measured_value
0	1	chamber_temp	398.0
1	1	chamber_temp	402.0
2	1	chamber_temp	405.0
3	4	chamber_temp	399.0
4	4	chamber_temp	401.0
5	7	chamber_temp	408.0
6	7	chamber_temp	416.0
7	7	chamber_temp	418.0

③ 조건 필터링

- 특정 조건에 해당하는 데이터만 선택한다.

- 예시 2. 특정 값 이상만 추출

```
high_temp_df = df[df['measured_value'] >= 410]  
print(high_temp_df)
```

	run_id	sensor_name	measured_value
6	7	chamber_temp	416.0
7	7	chamber_temp	418.0
16	2	rf_power	500.0
17	2	rf_power	508.0
18	5	rf_power	495.0
19	5	rf_power	505.0
20	8	rf_power	525.0
21	8	rf_power	530.0

④ 데이터 묶기 (groupby)

- Pandas DataFrame의 groupby() 함수
 - 같은 값을 가진 데이터끼리 묶어서 통계나 집계 연산을 수행하는 함수
 - 예) 센서별 평균값, 장비별 실행 횟수, 공정 그룹별 최대 온도

- 기본 구조 :

```
# 기본 구조  
df.groupby(기준컬럼)[대상컬럼].집계함수()
```

- 자주 사용하는 집계 함수 :

함수	의미
mean()	평균
max()	최대값
min()	최소값
count()	개수
sum()	합계
std()	표준편차

④ 데이터 묶기 (groupby)

- 예시 : 센서별 평균 측정값 계산

```
sensor_mean = df.groupby('sensor_name')['measured_value'].mean()

print(sensor_mean)
```

```
sensor_name
chamber_pressure      2.037500
chamber_temp         405.875000
chemical_temp         24.750000
cleaning_flow         20.500000
defect_count          3.500000
gas_flow             97.166667
inspection_temp       25.500000
rf_power             510.500000
Name: measured_value, dtype: float64
```

④ 데이터 묶기 (groupby)

- 예시 : 장비별 공정 실행 횟수

```
# 장비별 공정 실행 횟수
runcount_per_equip = df.groupby('equipment_id')['run_id'].count()
print(runcount_per_equip)
```

equipment_id	
101	16
102	12
103	4
104	4

⑤ 데이터 형태 변환하기 (pivot_table)

- Pandas DataFrame의 pivot_table() 함수
 - 세로(long) 형태의 데이터를 가로(wide) 형태의 분석용 표로 변환하는 함수
 - 동시에 그룹화, 요약(평균, 합계 등)도 함께 수행
- 기본 구조:

```
# 기본 구조
df.pivot_table(
    index=...,
    columns=...,
    values=...,
    aggfunc=...
)
```

파라미터	의미	예시
index	행 기준이 될 컬럼	'run_id'
columns	새로운 컬럼이 될 값	'sensor_name'
values	실제 값으로 사용할 컬럼	'measured_value'
aggfunc	같은 위치에 여러 값이 있을 때 계산 방법	'mean', 'max', 'min', 'count', 'sum'
fill_value	NaN 대신 채울 값	0
margins	합계 행/열 추가 여부	True

⑤ 데이터 형태 변환하기 (pivot_table)

- 예시 : 센서 종류를 컬럼으로 변환

```
pivot_df = df.pivot_table(  
    index='run_id',  
    columns='sensor_name',  
    values='measured_value'  
)  
  
print(pivot_df)
```

sensor_name	chamber_pressure	chamber_temp	chemical_temp	...	gas_flow	inspection_temp	rf_power
run_id				...			
1	2.0	401.666667	NaN	...	NaN	NaN	NaN
2	NaN	NaN	NaN	...	101.0	NaN	504.0
3	NaN	NaN	24.75	...	NaN	NaN	NaN
4	2.0	400.000000	NaN	...	NaN	NaN	NaN
5	NaN	NaN	NaN	...	99.5	NaN	500.0
6	NaN	NaN	NaN	...	NaN	25.5	NaN
7	2.1	414.000000	NaN	...	NaN	NaN	NaN
8	NaN	NaN	NaN	...	91.0	NaN	527.5

⑤ 데이터 형태 변환하기 (pivot_table)

- 예시 : 센서별 최대값

```
df.pivot_table(  
    index='run_id',  
    columns='sensor_name',  
    values='measured_value',  
    aggfunc='max'  
)
```

- 예시 : 장비별 센서 측정 횟수

```
df.pivot_table(  
    index='equipment_id',  
    columns='sensor_name',  
    values='measured_value',  
    aggfunc='count'  
)
```

⑥ 분석용 데이터셋 저장

- 정리된 데이터를 CSV 파일로 저장한다.

```
pivot_df.to_csv(  
    'clean_sensor_data.csv',  
    encoding='utf-8-sig'  
)
```

run_id	chamber_pressure	chamber_temp	chemical_temp	cleaning_flow	defect_count	gas_flow	inspection_temp	rf_power
1	2	401.6666667						
2						101		504
3			24.75	20.5				
4	2	400						
5						99.5		500
6					3.5		25.5	
7	2.1	414						
8						91		527.5

데이터 정제 실습 문제

① 원시 데이터 조회

문제 1

- RunHistory, Equipment, ProcessStep 테이블을 JOIN하여 공정 실행 이력 데이터를 조회하시오.
- 조회 항목:
 - run_id
 - equipment_id
 - model_name
 - step_name
 - process_group
 - run_status
 - start_time
- 조회 결과를 DataFrame으로 저장하고 앞 5개 행을 출력하시오.

데이터 정제 실습 문제

② 필요한 데이터 선택

문제 2

- 현재 DataFrame에서 다음 컬럼만 선택하여 새로운 DataFrame을 생성하시오.
- run_id, step_name, run_status
- 결과를 출력하시오.

문제 3

- 현재 DataFrame에서 다음 컬럼만 선택하여 새로운 DataFrame을 생성하시오.
- equipment_id, model_name, process_group
- 결과를 출력하시오.

데이터 정제 실습 문제

③ 조건에 맞게 정리

문제 4

- 공정 상태(run_status)가 'warning'인 데이터만 추출하시오.
- 결과를 새로운 DataFrame으로 저장하시오.

문제 5

- 공정 그룹(process_group)이 'ETCH'인 데이터만 조회하시오.
- 조회 결과를 출력하시오.

데이터 정제 실습 문제

④ 데이터 묶기 (groupby)

문제 6

- 공정 그룹별 공정 실행 횟수를 계산하시오. 출력 결과에는 다음 정보가 포함되어야 한다.
 - process_group
 - 공정 실행 횟수

문제 7

- 장비별 수행 공정 수를 계산하시오. 출력 결과에는 다음 정보가 포함되어야 한다.
 - equipment_id
 - model_name
 - 수행 공정 수

데이터 정제 실습 문제

⑤ 형태 변환 (pivot)

문제 8 다음 조건으로 pivot_table을 생성하고 결과를 출력하시오.

- index: equipment_id
- columns: process_group
- values: run_id
- aggfunc: count

문제 9 다음 조건으로 pivot_table을 생성하고 결과를 출력하시오.

- index: lot_id
- columns: run_status
- values: run_id
- aggfunc: count

데이터 정제 실습 문제

⑥ 분석용 데이터셋 생성

문제 10

- 다음 조건에 맞는 공정 실행 분석용 데이터셋을 생성하시오.
 - equipment_id 기준으로 데이터 구성
 - process_group을 컬럼으로 변환
 - 수행 횟수를 값으로 사용
 - 결과를 CSV 파일로 저장
 - 파일 이름: equipment_process_summary.csv
 - UTF-8 인코딩 적용



끝!

